



Human Interface Guidelines

The Complete Unabridged Reference Manual

Full-fidelity compilation of Apple UI foundational design systems, patterns, and specifications.

1. Foundations & Accessibility

Accessibility

Accessible user interfaces empower everyone to have a great experience with your app or game. When you design for accessibility, you reach a larger audience and create a more inclusive experience. An accessible interface allows people to experience your app or game regardless of their capabilities or how they use their devices. Accessibility makes information and interactions available to everyone.

- **Intuitive:** Your interface uses familiar and consistent interactions that make tasks straightforward to perform.
- **Perceivable:** Your interface doesn't rely on any single method to convey information. People can access and interact with your content, whether they use sight, hearing, speech, or touch.
- **Adaptable:** Your interface adapts to how people want to use their device, whether by supporting system accessibility features or letting people personalize settings.

Audit the accessibility of your interface using the Accessibility Inspector to highlight accessibility issues with your interface and to understand how your app represents itself to people using system accessibility features. You can also communicate how accessible your app is on the App Store using Accessibility Nutrition Labels.

Vision

The people who use your interface may be blind, color blind, or have low vision or light sensitivity. They may also be in situations where lighting conditions and screen brightness affect their ability to interact with your interface.

Support larger text sizes: Make sure people can adjust the size of your text or icons to make them more legible, visible, and comfortable to read. Ideally, give people the option to enlarge text by at least 200 percent (or 140 percent in watchOS apps). Your interface can support font size enlargement either through custom UI, or by adopting Dynamic Type. Dynamic Type is a systemwide setting that lets people adjust the size of text for comfort and legibility.

Use recommended defaults for custom type sizes: Each platform has different default and minimum sizes for system-defined type styles to promote readability. If you're using custom type styles, follow the recommended defaults:

Platform	Default size	Minimum size
iOS, iPadOS	17 pt	11 pt
macOS	13 pt	10 pt
tvOS	29 pt	23 pt
visionOS	17 pt	12 pt
watchOS	16 pt	12 pt

Bear in mind that font weight can also impact how easy text is to read. If you're using a custom font with a thin weight, aim for larger than the recommended sizes to increase legibility. Thicker weights are easier to read for smaller font sizes. Consider increasing the font size when using a thin weight.

Strive to meet color contrast minimum standards: To ensure all information in your app is legible, it's important that there's enough contrast between foreground text and icons and background colors. Two

popular standards of measure for color contrast are the Web Content Accessibility Guidelines (WCAG) and the Accessible Perceptual Contrast Algorithm (APCA). Use standard contrast calculators to ensure your UI meets acceptable levels. Accessibility Inspector uses the following values from WCAG Level AA as guidance in determining whether your app's colors have an acceptable contrast:

Text size	Text weight	Minimum contrast ratio
Up to 17 pts	All	4.5:1
18 pts	All	3:1
All	Bold	3:1

If your app doesn't provide this minimum contrast by default, ensure it at least provides a higher contrast color scheme when the system setting Increase Contrast is turned on. If your app supports Dark Mode, make sure to check the minimum contrast in both light and dark appearances.

Incorrect: A button with insufficient color contrast between title and background makes content unreadable.

Correct: A button with sufficient color contrast meets WCAG requirements and ensures readability.

Prefer system-defined colors: These colors have their own accessible variants that automatically adapt when people adjust their color preferences, such as enabling Increase Contrast or toggling between the light and dark appearances (e.g., systemRed default vs accessible variants in iOS).

Convey information with more than color alone: Some people have trouble differentiating between certain colors and shades. For example, people who are color blind may have particular difficulty with pairings such as red-green and blue-orange. Offer visual indicators, like distinct shapes or icons, in addition to color to help people perceive differences in function and changes in state. Consider allowing people to customize color schemes such as chart colors or game characters so they can personalize your interface in a way that's comfortable for them.

Incorrect: Using only color (e.g., a green circle and red circle) to show status. For someone with red-green color blindness, these indicators might appear the same.

Correct: Using both shapes and color (e.g., a green circle containing a checkmark next to a red octagon containing an X) to clearly help differentiate differences.

Describe your app's interface and content for VoiceOver: VoiceOver is a screen reader that lets people experience your app's interface without needing to see the screen.

Hearing

The people who use your interface may be deaf or hard of hearing. They may also be in noisy or public environments.

Support text-based ways to enjoy audio and video: It's important that dialogue and crucial information about your app or game isn't communicated through audio alone. Depending on the context, give people

different text-based ways to experience their media, and allow people to customize the visual presentation of that text:

- **Captions** give people the textual equivalent of audible information in video or audio-only content. Captions are great for scenarios like game cutscenes and video clips where text synchronizes live with the media.
- **Subtitles** allow people to read live onscreen dialogue in their preferred language. Subtitles are great for TV shows and movies.
- **Audio descriptions** are interspersed between natural pauses in the main audio of a video and supply spoken narration of important information that's presented only visually.
- **Transcripts** provide a complete textual description of a video, covering both audible and visual information. Transcripts are great for longer-form media like podcasts and audiobooks where people may want to review content as a whole or highlight the transcript as media is playing.

Use haptics in addition to audio cues: If your interface conveys information through audio cues — such as a success chime, error sound, or game feedback — consider pairing that sound with matching haptics for people who can't perceive the audio or have their audio turned off. In iOS and iPadOS, you can also use Music Haptics and Audio graphs to let people experience music and infographics through vibration and texture.

Augment audio cues with visual cues: This is especially important for games and spatial apps where important content might be taking place off screen. When using audio to guide people towards a specific action, also add in visual indicators that point to where you want people to interact.

Mobility

Ensure your interface offers a comfortable experience for people with limited dexterity or mobility.

Offer sufficiently sized controls: Controls that are too small are hard for many people to interact with and select. Strive to meet the recommended minimum control size for each platform to ensure controls and menus are comfortable for all when tapping and clicking.

Platform	Default control size	Minimum control size
iOS, iPadOS	44x44 pt	28x28 pt
macOS	28x28 pt	20x20 pt
tvOS	66x66 pt	56x56 pt
visionOS	60x60 pt	28x28 pt
watchOS	44x44 pt	28x28 pt

Consider spacing between controls as important as size: Include enough padding between elements to reduce the chance that someone taps the wrong control. In general, it works well to add about 12 points of padding around elements that include a bezel. For elements without a bezel, about 24 points of padding works well around the element's visible edges.

Incorrect: Controls or buttons grouped closely together with insufficient padding can easily cause mistaps.

Correct: Controls spaced out with sufficient padding around them guarantee precision and easy targeting.

Support simple gestures for common interactions: For many people, complex gestures can be challenging. Use the simplest gesture possible — avoid custom multifinger and multihand gestures — so repetitive actions

are both comfortable and easy to remember. Offer alternatives to gestures. Gestures can be less comfortable for people who have limited dexterity, so offer onscreen ways to achieve the same outcome (e.g., provide an edit button and a tap-to-delete interface as an alternative to swipe-to-delete layouts).

Voice Control, Siri & Assistive Technologies: Let people use Voice Control to give guidance and enter information verbally by labeling interface elements appropriately. Integrate with Siri and Shortcuts to let people perform repetitive tasks using voice alone from Siri, the Action button, or Home Screen shortcuts. Support mobility-related assistive technologies like AssistiveTouch, Full Keyboard Access, Pointer Control, and Switch Control by ensuring clear labels and performing comprehensive testing.

Speech, Cognitive & visionOS Considerations

Speech Accommodations: Let people use the keyboard alone to navigate and interact with your app. Avoid overriding system-defined keyboard shortcuts and ensure full keyboard navigation support. Support Switch Control so users can control devices through separate hardware, game controllers, or sound events (such as clicks or pops).

Cognitive Accommodations: Keep actions simple and intuitive. Prefer familiar system gestures over custom ones. Minimize the use of time-boxed interface elements; views and controls that auto-dismiss on a timer can be problematic for people who need longer to process information or traverse UI paths. Prefer dismissing views with an explicit action. In games, consider offering difficulty accommodations (adjusting reaction times, control assistance). Control audio/video playback explicitly and do not autoplay without controls. Allow users to opt out of flashing lights during video playback. Reduce fast-moving or blinking animations when Reduce Motion is active (tighten animation springs to reduce bounce, track animations directly with gestures, avoid depth shifts on the z-axis, replace transitions with alpha fades, and avoid animating into/out of blurs).

Optimize for Assistive Access: Streamline your app for iOS Assistive Access mode. Identify core functionality and remove noncritical flows, break up multistep workflows into single interactions per screen, and always ask for confirmation twice whenever people perform an action that's difficult to recover from, such as deleting a file.

visionOS Comfort Guidelines: Immersive environments carry a higher risk of motion sickness, visual fatigue, and ergonomic strain. Keep elements within the core field of view. Prefer horizontal layouts over tall vertical ones that cause neck strain. Reduce speed/intensity of animations, especially in peripheral vision. Avoid anchoring content directly to the wearer's head (anchor content in space instead so users can look around naturally). Minimize the need for massive, repetitive gestures that cause physical fatigue.

2. Visual Design: Color, Dark Mode, Icons & Images

Color Best Practices & Inclusive Color

Avoid using the same color to mean different things: Use color consistently throughout your interface, especially when you use it to help communicate information like status or interactivity. Stylizing noninteractive text with the same color used for interactive brand buttons causes serious layout confusion.

Adapt to Contexts: Make sure all your app's colors work well in light, dark, and increased contrast contexts. Supply specific light, dark, and high-contrast variants for custom colors if you don't use system colors, which adapt automatically. Test schemes under extreme lighting conditions (muted in bright sunlight, saturated in dark rooms). Account for True Tone display color shifting and hardware P3 vs sRGB color profiles. Avoid relying solely on color to communicate essential metrics or states; pair colors with explicit text labels or glyph shapes. Consider localized color perceptions (e.g., green implies positive stock trends in English markets, whereas red indicates a positive rising trend in Chinese stock apps).

System Color Guidelines: Avoid hard-coding system color values. Dynamic system colors are semantically defined by purpose rather than absolute hex values (e.g., label, secondaryLabel, separator). Avoid redefining these semantic meanings (do not use a separator color as text font colors).

Liquid Glass Color: Liquid Glass has no inherent color by default and takes on characteristics of background content. Applying color tints to glass backgrounds must be done sparingly, reserving color only for prominent actions (e.g., a blue "Done" button) to build an obvious visual hierarchy. Do not apply color backgrounds to multiple adjacent toolbar components.

Incorrect: Applying a bright accent color tint to the backgrounds of all adjacent buttons in a toolbar row.

Correct: Keeping secondary buttons monochromatic and only applying the prominent accent tint to the primary action button.

Color Specifications

System Colors (SwiftUI APIs)

Color Name	Default Light (RGB)	Default Dark (RGB)	High Contrast Light	High Contrast Dark
Red	255, 56, 60	255, 66, 69	233, 21, 45	255, 97, 101
Orange	255, 141, 40	255, 146, 48	197, 83, 0	255, 160, 86
Yellow	255, 204, 0	255, 214, 0	161, 106, 0	254, 223, 67
Green	52, 199, 89	48, 209, 88	0, 137, 50	74, 217, 104
Mint	0, 200, 179	0, 218, 195	0, 133, 117	84, 223, 203
Teal	0, 195, 208	0, 210, 224	0, 129, 152	59, 221, 236
Cyan	0, 192, 232	60, 211, 254	0, 126, 174	109, 217, 255
Blue	0, 136, 255	0, 145, 255	30, 110, 244	92, 184, 255
Indigo	97, 85, 245	109, 124, 255	86, 74, 222	167, 170, 255
Purple	203, 48, 224	219, 52, 242	176, 47, 194	234, 141, 255
Pink	255, 45, 85	255, 55, 95	231, 18, 77	255, 138, 196
Brown	172, 127, 94	183, 138, 102	149, 109, 81	219, 166, 121

iOS, iPadOS System Gray Colors

Gray Name	Default Light (RGB)	Default Dark (RGB)	High Contrast Light	High Contrast Dark
systemGray	142, 142, 147	142, 142, 147	108, 108, 112	174, 174, 178
systemGray2	174, 174, 178	99, 99, 102	142, 142, 147	124, 124, 128
systemGray3	199, 199, 204	72, 72, 74	174, 174, 178	84, 84, 86
systemGray4	209, 209, 214	58, 58, 60	188, 188, 192	68, 68, 70
systemGray5	229, 229, 234	44, 44, 46	216, 216, 220	54, 54, 56
systemGray6	242, 242, 247	28, 28, 30	235, 235, 240	36, 36, 38

Dark Mode

Dark Mode uses a dark color palette for all elements, tailored for low-light environments. **Avoid offering an app-specific appearance setting;** an app-specific mode creates redundant adjustments and breaks consistency. Ensure apps adapt dynamically if users set appearance to Auto. Check legibility with Increase Contrast and Reduce Transparency active. Soften white background layers on image content assets to stop them from glowing intensely within dark contexts. Maintain a contrast ratio of at least 4.5:1 (7:1 for custom small text blocks).

In iOS and iPadOS, the system uses two sets of background colors—**Base** and **Elevated**—to enhance depth layout tracking when dark windows overlay one another. Base options are dimmer to let background frames recede, while elevated layers are brighter, causing active modal sheets and foreground content to advance cleanly.

Icons & SF Symbols

Interface icons should use a simplified, recognizable design based on familiar metaphors. Maintain stroke thickness, scale, and optical weights consistently across all graphics. Adjust asymmetric paths manually so that they are optically centered within geometry footprints rather than strictly mathematically centered. Provide a selected-state alternate icon only if necessary (the system tints toolbar/tab items automatically). Avoid text strings inside icons unless essential; make sure to localize individual layout characters if used. Deliver custom assets in clean vector maps (PDF/SVG).

SF Symbols integrates natively with San Francisco text baselines. It supports four rendering modes: Monochrome (single tint), Hierarchical (single color with varied opacities per layer), Palette (multiple explicit colors per path layer), and Multicolor (intrinsic real-world colors like green leaves or red delete markers). Variable color scales can be assigned to reflect quantitative value changes over time (like connection steps or battery tiers). Symbol animations (Bounce, Pulse, Scale, Replace, Breathe, Rotate, Variable Color, Draw On/Off) should be applied purposefully to communicate state shifts cleanly without distracting users.

macOS Custom Document Icons: Document icons feature a folded top-right corner. You can supply a background fill, a center image, and short text strings. The canvas scales from 16x16 px up to 1024x1024 px (@2x). Keep important elements fully away from the top-right corner mask boundary to prevent them from being covered by the system-drawn white paper fold asset.

Images & Scale Factors

Images require high-resolution asset variants tailored to platform scale densities. Points represent an abstract measurement unit. In 2D displays, points map to physical pixels based on device resolutions; in visionOS, points represent an angular value scaling cleanly based on real-time distances from the viewer.

Platform Workspace	Required Asset Scale Tiers
iPadOS, watchOS	@2x
iOS	@2x and @3x
macOS, tvOS	@1x and @2x
visionOS	@2x or higher (vector preferred)

tvOS Parallax & Layered Images: tvOS app icons require layered images consisting of 2 to 5 distinct layout paths. Foreground layers hold sharp elements (text or characters), middle zones contain drop shadows or accessory art, and background vectors must remain completely opaque. When a user focuses on a layered target, the platform applies a dynamic parallax tilt movement to add life and visual value.

visionOS Imagery: visionOS dynamically resizes image asset resolutions. Vector paths are heavily preferred for 2D graphics. Spatial photos utilize stereoscopic HEIC metadata containers. To display readable text blocks directly over spatial photos, use a feathered glass background canvas effect to blur complex visual noise safely and elevate text legibility.

3. Design Patterns & System Behaviors

Inclusion & Inclusive Writing

Design apps with empathy, avoiding culture-exclusive or stereotypical assumptions across age, gender, race, disability, or financial standing. Write software copy directly, addressing people using welcoming words like "you" and "your" rather than distant references like "the user" or "the player". Define any technical jargon strings clearly. Avoid culture-bound colloquialisms (e.g., avoid historical or regional phrases like "grandfathered in" or "peanut gallery"). Use humor cautiously since it doesn't translate uniformly and can feel repetitive or annoying over time. Represent human diversity naturally across occupations and configurations. Avoid traditional binary restrictions if gathering gender fields; provide inclusive options like nonbinary, self-identify, or decline to state alongside clear personal pronoun choices.

Layout, Visual Hierarchy & Adaptability

Group related elements cleanly using whitespace bounding, distinct backgrounds, materials, or border lines. Structure visual flows based on natural reading directions (top-to-bottom, leading-to-trailing edge), taking RTL localizations into account. Use progressive disclosure to reveal detailed settings groups only when contextually relevant. Support dynamic layout engine systems (SwiftUI / Auto Layout) to gracefully adjust to orientation changes, window resizing actions, dynamic type enlargement tiers, or localized text expansion requirements. Always strictly respect layout safe areas to guarantee that active app content never gets obstructed by physical device features, like the Dynamic Island or camera housing bars.

Materials, Depth & Motion

Materials separate layers visually by letting background colors bleed through to construct structure. In iOS and iPadOS, standard materials inside the content layer consist of four density tiers: ultra-thin, thin, regular, and thick. Pair materials with system vibrant label styles to protect text contrast levels. In visionOS, windows use adaptive system glass texturing that modifies its illumination based on physical surroundings; keep window assets translucent to maintain spatial awareness for the wearer. Apply spatial depth to establish information hierarchies (e.g., pushing a sheet forward along the z-axis), but avoid placing raw depth dimensions onto body text paths because hovering characters are difficult to read and cause visual fatigue.

Motion must remain fluid, short, precise, and entirely purposeful. Always ensure animations are fully optional by offering alternative visual cues or audio/haptic responses. Users must be able to cancel or interrupt ongoing animations instantly without lagging or freezing the software state.

Privacy & Data Protection Protocols

Request data authorization parameters only when a feature explicitly requires them. Provide highly transparent purpose strings inside configuration alerts. These descriptions must use sentence-style casing, an active voice, and clear terminal punctuation.

Incorrect: "Microphone access is needed for a better experience." (Vague, passive voice, zero contextual reasoning).

Correct: "The app records during the night to detect snoring sounds." (Direct, active voice, functional explanation).

Do not show permission alerts during the initial app launch flow unless the feature is required for core operations. Pre-alert screens used to detail data advantages must feature only a single button labeled "Continue" or "Next" to transition directly to the system prompt; do not inject custom cancel actions or manipulate choices. Prohibit any tracking incentive designs, alert replication images, or artificial layout blockades that push users to opt into system tracking, as this will lead to immediate App Store review rejections. Safeguard sensitive records locally within the system keychain; never store passwords or identity strings in plain text files.

Right to Left (RTL) Localizations

Flip layout directions completely in RTL scripts like Arabic or Hebrew. Text blocks spanning more than three lines should align based on their source language rather than the system layout direction to keep paragraphs highly legible. Never reverse the digit sequence of phone numbers, credit cards, or standard numeric values (e.g., "12345" stays "12345" across all scripts). Flip numeric progress controls, volume trackers, or sliders alongside accompanying endpoint glyph locations. In buttons or titles, increase Arabic or Hebrew font scales by roughly 2 points to visually balance them against capitalized Latin characters. Avoid horizontally flipping actual illustrations or photographs, but ensure interface icons indicating directionality, text structures, or forward motion are flipped correctly.

4. Interface Views, Menus & Controls

Windows, Sheets, Popovers & Modality

Windows: Primary windows host an app's core content, while auxiliary windows focus on standalone tasks. Windows must dynamically resize to accommodate side-by-side multitasking views. In macOS, main windows receive color inside window controls, while inactive frames drop to flat gray tints and lose transparency effects. Keep window bottoms clean from essential commands as users often drag window frames partially below screen bounds.

Sheets: Sheets manage targeted subtasks. They can be modal or nonmodal (allowing parent interaction, like the Notes formatting panel). Single-view sheets place "Cancel" on the leading edge and "Done" on the trailing edge of the top toolbar. For resizable sheets, include a visible grabber bar to show resizability and support VoiceOver commands. In macOS, sheets float as card assets, dimming the parent window while letting other app windows stay interactive.

Popovers & Modality: Popovers are transient views extending arrows toward the parent control that triggered them. Do not cascade popovers or stack multiple popover layers together. Modality isolate users from the parent context to force focus or request decisions. Keep modal paths short and streamlined; avoid complex view hierarchies that make users lose track of their original context.

Menus, Bars & Key Navigation Elements

Menus & Pop-up/Pull-down Controls: Menus list commands using title-style capitalization. Remove articles ("a", "an", "the") to protect horizontal space. Display trailing ellipses (...) if an action demands extra choices or text inputs before completion. Pop-up buttons display mutually exclusive options and update the button face to track selections. Pull-down buttons host context-related operations or action shortcuts (e.g., adding or sorting fields); if a pull-down action is destructive, format the item text in red.

The Menu Bar (macOS & iPadOS): Displays core commands in a fixed system hierarchy: App Menu, File, Edit, Format, View, App-Specific items, Window, and Help. Always disable unavailable menu choices rather than hiding them so users can learn the app's full capabilities. Undo and Redo commands should append context string references dynamically (e.g., "Undo Typing").

Sidebars & Tab Bars: Sidebars provide top-level navigation on the leading edge of a view. Tab bars switch sections at the bottom of the screen (iOS) or top edge (iPadOS). Do not hide or disable tab options if their specific view is empty—instead, use the layout to explain why content is currently missing. Use badges sparingly to highlight critical updates.

Data Entry, Fields, Toggles & Indicators

Text Fields & Entry Views: Text fields receive specific data inputs. Match field box widths to the volume of expected text strings. Provide a clear button on the trailing side to let users wipe inputs instantly. Set the virtual keyboard type semantically (e.g., email address, decimal pad) to match tasks perfectly. Use secure text entry fields to automatically mask sensitive content like passwords with dot indicators.

Toggles, Steppers & Sliders: Toggles switch between opposing states (On/Off). Use the switch layout style exclusively inside list rows; outside lists, prefer buttons that change selection states. Sliders track values across

a continuous range from a leading minimum point to a trailing maximum point. Steppers provide two button segments to increment or decrement whole values precisely.

Progress & Rating Indicators: Progress views indicate ongoing operations. Use determinate tracks when duration is calculated, and shifting indeterminate spinners when operations are unquantifiable. Switch indicators from indeterminate to determinate fields once durations can be calculated. Rating indicators line up symbols horizontally (stars by default) to communicate fixed ranking levels clearly.

Gauges, Charts & Specialized Layouts

Gauges & Activity Rings: Gauges display a specific numerical value within a defined range using circular or linear tracks. Activity Rings track daily Move, Exercise, and Stand progression using fixed crimson, lime, and cyan tracks. Activity rings must always render over a pure black background and should never be used as general visual decorations or branding elements.

Charts: Charts map data points visually into a clean plot area using bar marks (comparing categories), line marks (tracking shifts over time), or point marks (inspecting distributions). Axis limits can be fixed or dynamic. Avoid over-crowding the plot canvas with too many grid lines or dense labels. Ensure charts are accessible by providing deep VoiceOver description labels for marks and adopting Audio Graphs to generate descriptive audio tones tracking data shifts dynamically.

App Shortcuts: Surface up to 10 key functions natively within Spotlight, Siri, and system nodes using App Intents. Keep verbal phrases simple and intuitive. Use title case and plural configurations when referencing the Shortcuts system tool.